Shapes-VQA Hard: How to Use It

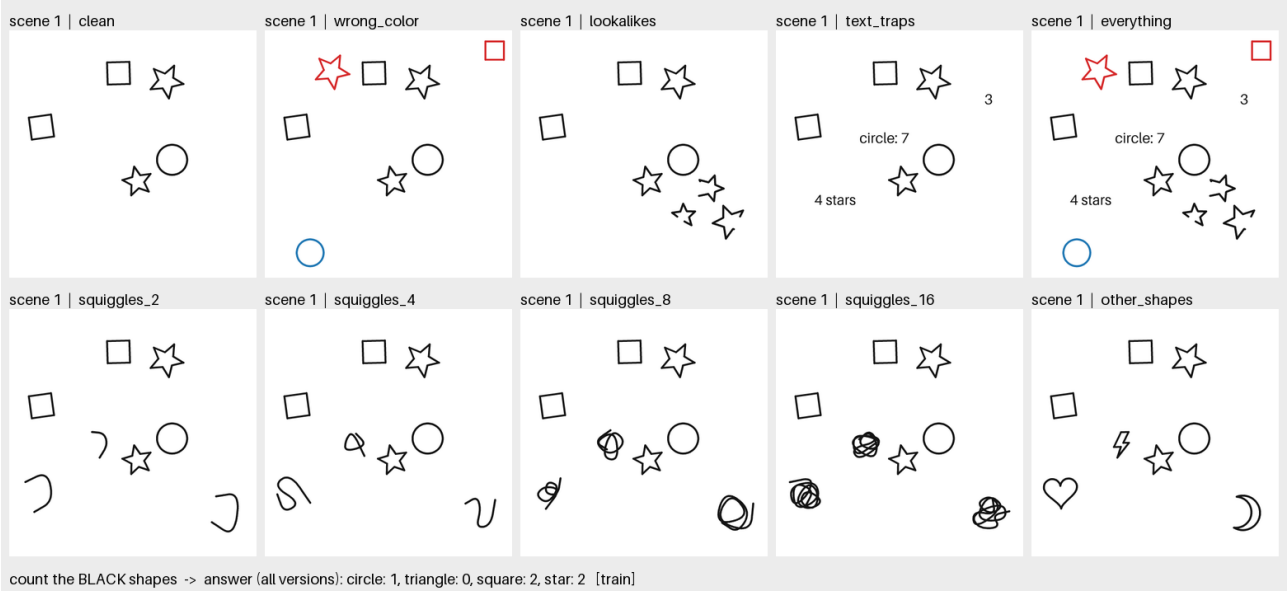
ATML project | huggingface.co/datasets/atml-shapes-vqa/shapes-vqa-hard | questions: ask Mohid

What it is

The harder version of our dataset: count only the named shapes (circle, triangle, square, star) **of one colour**, named in the question. **500 scenes x 10 versions = 5,000 images**. The traps target **mixing up colours and shapes**, **being fooled by text in the image**, **squiggles of growing complexity**, and **clutter with a name vs. clutter without one**.

Version	Diff.	What is added to the target-colour shapes	What it tests
clean	0	nothing	baseline
wrong_color	1	perfect named shapes in **other** colours	colour-shape binding
lookalikes	2	broken or wavy shapes in the **target** colour	shape similarity
text_traps	3	misleading text like "star: 4", "red circle", "5" (count claims are always wrong)	being fooled by text
everything	4	all traps from versions 1, 2 and 3 together	worst case
squiggles_2, _4, _8, _16	5-8	scribbles with 2, 4, 8 or 16 curves (a curve = one bend of 30-120 degrees), in the **same spots, sizes and colours** in all four	squiggle complexity
other_shapes	9	**nameable** shapes that are none of the four types (heart, moon, cross, lightning bolt), in **the squiggle spots, sizes and colours**	clutter with vs. without a name

Key idea: the target shapes are pixel-identical in all 10 versions of a scene, so **all 10 versions have the same correct answer**. Each version adds one kind of trap, so each version's accuracy drop tells you **which trap** hurts the model most. The 4 squiggle versions change **only** the number of curves, so they give accuracy vs. complexity. `other_shapes` draws nameable shapes in exactly the squiggle spots, so comparing it with the squiggle versions isolates **whether the clutter has a name**.



The question is the same for every image except the colour (red, blue, green or black):

```
... Only count complete, regular shapes that are red and are one of these four types:
circle, triangle, square, star. Do not count shapes of any other colour, broken or wavy
shapes, or any words or numbers written in the image. Answer in exactly this format: ...

answer:  circle: 1, triangle: 0, square: 2, star: 1      (all 4 classes, zeros included)
```

Versions 1-3, each squiggle version and `other_shapes` add 2, 3 or 4 traps; *everything* has exactly the union of versions 1-3 (6, 9 or 12). None of the other shapes is or contains a circle, triangle, square or star, so they are never counted. Splits are by scene: **train** 3,500 images (350 scenes), **validation** 750 (75), **test** 750 (75). `other_shapes` was added in v1.2; the other 9 versions are byte-identical to before, so results you already have stay valid.

Useful columns

Column	Meaning
`image_id`	e.g. hard_0042_text_traps. The key for your predictions
`target_color`	the colour to count: red / blue / green / black
`scene_id`	same for all 10 versions of a scene. Use it to compare versions
`version, difficulty`	clean / wrong_color / lookalikes / text_traps / everything = 0-4, squiggles_2 / _4 / _8 / _16 = 5-8, other_shapes = 9
`count_<shape>, count_total`	true counts (target colour only)
`objects`	JSON list of every object; traps have role = distractor. Lookalikes have lookalike_of and subtype gap / wavy, text traps the written text, squiggles curves and turning_deg (total bending), other shapes shape = heart / moon / cross / bolt

Step 1. Get access (skip if you already did this for the other datasets)

1. Join the **atml-shapes-vqa** organization on Hugging Face (Mohid sends the invite link). 2. At **huggingface.co/settings/tokens** create a token of type Read. 3. In a terminal:

```
pip install datasets huggingface_hub pillow
hf auth login          # paste your token (older versions: huggingface-cli login)
```

Step 2. Load the data

```
from datasets import load_dataset
ds = load_dataset("atml-shapes-vqa/shapes-vqa-hard")
test = ds["test"]            # 750 images = 75 scenes x 10 versions
ex = test[0]
ex["image"]          # PIL image, 512 x 512
ex["target_color"]   # e.g. "red"
ex["question"]       # the prompt (already names the colour)
ex["answer"]         # "circle: 1, triangle: 0, square: 2, star: 1"
ex["version"]        # clean / wrong_color / ... / squiggles_16 / other_shapes
text_only = test.filter(lambda e: e["version"] == "text_traps")   # one version only
```

Step 3. Run your model and save its answers

Same as the other datasets: a CSV of `image_id, prediction`. Always pass the given question (it names the colour).

```
import csv
def ask_model(image, question):
    ...   # your VLM here (Qwen2-VL, LLaVA, InternVL, ...). Return its text answer.

with open("preds_hard_mymodel.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["image_id", "prediction"])
    for ex in test:
        w.writerow([ex["image_id"], ask_model(ex["image"], ex["question"])])
```

Step 4. Score it

```
# once: download the scoring script (the same script works for all our datasets)
hf download atml-shapes-vqa/shapes-vqa-hard vqa_scoring.py --repo-type dataset --local-dir .

# score: it sees the "hard_" image ids and fetches the right answers by itself
python vqa_scoring.py --predictions preds_hard_mymodel.csv --split test
```

You get one row per version: **exact** (all 4 counts right), per-class accuracy, **abs_err** (summed count error), **overcnt** (share of images where the model counted more target shapes than there are) and **parsed**, plus a **paired comparison against clean** with a McNemar p-value. Add `--out scores.csv` to get the per-image scores.

How to read the results

- Big drop on `wrong_color`: the model mixes up colours and shapes (binding problem).
- Big drop on `lookalikes`: it can't tell near-miss shapes from real ones.
- Big drop on `text_traps`: it reads the text and copies its numbers (the written words are in `objects`).
- `everything`: do the traps add up, or is one trap enough to break it?
- `squiggles_2` to `squiggles_16`: plot accuracy against the number of curves. A steady fall means more complex clutter hurts more, even though nothing else changes.
- `other_shapes` vs. the squiggle versions: same spots, sizes, colours and number of traps; only whether the clutter has a name differs. A bigger drop on other_shapes means clutter the model can name distracts it more. Compare on the same scenes (paired by scene_id).

Rules of thumb

- **Report results on the test split**; use train / validation for anything you tune. **Use the question as given**, so the 10 versions stay comparable. If something looks wrong, tell Mohid.